



MACHINE LEARNING MINI RESEARCH

SIT307 – Machine Learning



SEPTEMBER 28, 2026
HUYNH QUANG VINH (KENT)
224672479

Contents

1. Published Study and Research Context.....	1
1.1 Research Problem and Motivation	1
1.2 State-of-the-Art and Research Gap	1
1.3 Contribution of the Selected Research Paper	1
2. Dataset and Feature Set Used.....	2
2.1 Dataset and Features	2
2.2 Data Inspection.....	3
2.3 Preprocessing and Reproduction Assumption.....	3
3. Reproduction Methodology	4
3.1 Individual Classifiers.....	4
3.2 Stacking Ensemble	5
3.3 Evaluation Metrics	5
4. Reproduction Results	5
4.1 Individual Classifier Results	5
4.2 Stacking Ensemble Results	6
4.3 Overall Research Paper against Reproduced Comparison.....	6
5. Critical Analysis.....	7
5.1 Duplicated Records and Train-Test Overlap.....	7
5.2 Unique Dataset Experiment	7
6. Proposed Machine Learning Solution	9
6.1 Motivation and Identified Limitation.....	9
6.2 Proposed Methodology	9
6.3 Experimental Protocol.....	10
6.4 Experimental Results	11
6.5 Comparison	13
6.6 Critical Evaluation of the Proposed Method.....	13
6.7 Limitations of the Proposed Method	14
7. Conclusion	14
8. GitHub and Video Link	14
Acknowledgement.....	15
References.....	15

1. Published Study and Research Context

1.1 Research Problem and Motivation

Nowadays, heart disease remains a major health problem, so early detection can support preventive treatment and improve patient outcomes. Traditional approaches to cardiovascular risk assessment may not always accurately represent individual risk. Therefore, machine learning can provide an alternative approach for identifying patterns with heart disease [1].

The selected research paper by Bhagat et al. investigates heart disease prediction by using 6 machine learning classifiers and a stacking ensemble method. The main objective is to improve the prediction performance by combining the output of several classifiers via a meta-classifier [1].

1.2 State-of-the-Art and Research Gap

Previous research has applied different machine learning and feature selection approaches to heart disease prediction. These include Random Forest combined with linear models, CHI-PCA feature reduction, conventional classifiers such as Logistic Regression, Decision Tree, Random Forest, Naive Bayes, KNN and stacking ensemble approach [1]. However, the selected paper identifies challenges related to limited medical datasets, features selection, selection of machine learning algorithms and insufficient analysis [1].

Approach	Technique	Reported result
Mohan et al.	RF + Linear Model	88.47%
Garate-Escamila et al.	CHI-PCA + RF	98.7–99.4% depending on dataset
Ramesh et al.	SVM, KNN, DT, RF, NB	RF 88.35%
Latha et al.	Majority-vote ensemble	85.48%

Based on these limitations, the selected study investigates whether combining different classifiers via stacking can provide more reliable heart disease predictions. The proposed method integrates 6 individual models through 5-fold stacking ensemble instead of depending on 1 classifier [1].

1.3 Contribution of the Selected Research Paper

The selected study makes 3 main contributions relevant to this reproduction. First, it uses a public heart disease dataset, which contains data from multiple resources. Then, 6 classifiers including Logistic Regression, Decision Tree, Random Forest, XGBoost, Naive Bayes and KNN are evaluated individually. Finally, these classifiers are combined using 5-fold stacking ensemble to improve prediction performance [1].

2. Dataset and Feature Set Used

2.1 Dataset and Features

```
[5]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
 #   Column      Non-Null Count  Dtype  
---  --
 0   age         1025 non-null   int64   
 1   sex         1025 non-null   int64   
 2   cp          1025 non-null   int64   
 3   trestbps    1025 non-null   int64   
 4   chol        1025 non-null   int64   
 5   fbs         1025 non-null   int64   
 6   restecg     1025 non-null   int64   
 7   thalach     1025 non-null   int64   
 8   exang       1025 non-null   int64   
 9   oldpeak     1025 non-null   float64  
10  slope       1025 non-null   int64   
11  ca          1025 non-null   int64   
12  thal        1025 non-null   int64   
13  target      1025 non-null   int64   
dtypes: float64(1), int64(13)
memory usage: 112.2 KB
```

Figure 1. Dataset structure and feature data types of the Heart Disease dataset.

The reproduction uses the same public Heart Disease dataset from the selected research paper [1]. The original dataset includes 1025 records and 14 attributes, which also contains 13 features and 1 binary target variable. The target value 0 represents no heart disease, while 1 represents heart disease.

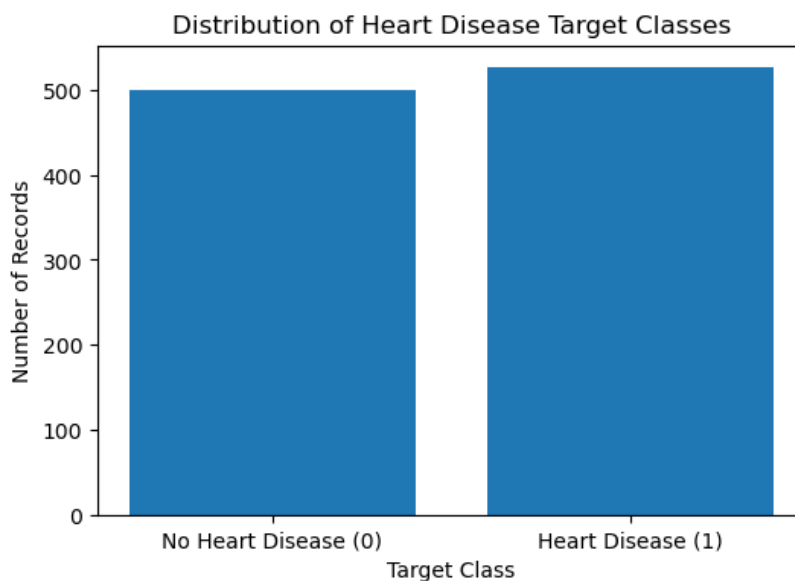


Figure 2. Distribution of heart disease target classes in the original dataset.

The target distribution is quite balanced, with 48.68% of negative cases and 51.23% positive cases. Therefore, substantial class imbalance is not observed in the provided dataset.

2.2 Data Inspection

After reviewing the provided dataset, there is no any missing value that is identified, although the research paper includes missing value handling as part of preprocessing [1].

```
[6]: df.isnull().sum()
```

```
[6]: age      0
     sex      0
     cp      0
     trestbps  0
     chol     0
     fbs      0
     restecg   0
     thalach   0
     exang     0
     oldpeak   0
     slope     0
     ca        0
     thal      0
     target    0
     dtype: int64
```

Figure 3. Missing-value inspection of the Heart Disease dataset.

Then, all categorical variables are already represented numerically in the original dataset, so additional encoding is not required.

Most feature ranges match the values demonstrated in the research paper. However, the original dataset contains $ca = 4$ and $thal = 0$, while the research paper demonstrates ca as $0 - 3$ and $thal$ as $1 - 3$ [1]. These values are retained because the research paper uses all 1025 records of the dataset without removing.

```
[7]: df.duplicated().sum()
```

```
[7]: 723
```

Figure 4. Exact duplicate count identified in the original dataset.

A total of 723 duplicated rows is also identified. These duplicated rows are also retained for the main reproduction to preserve the dataset size used in the research paper.

2.3 Preprocessing and Reproduction Assumption

The research paper first demonstrates missing-value handling, categorical encoding and feature scaling, which are followed by the train test split [1]. However, several

implementation details are not fully specified. Therefore, reason assumptions are applied to complete the reproduction.

Component	Paper information	Reproduction decision
Missing values	Missing-value handling	No imputation because no missing values is present
Categorical encoding	Encoding	No additional encoding because variables are already numerical
Scaling	Scaling/normalisation	StandardScaler
Train-test ratio	Not explicitly stated	80:20
Random seed	Not stated	random_state = 42
Test size	Not explicitly stated	205 samples

The 80:20 split is inferred from the paper stacking confusion matrix, which contains 205 test observations. Because the original dataset contains 1025 records, 205 corresponds to 20% of the dataset. Then, the exact random seed is not included in the research paper, so random_state = 42 is used for reproduction. Finally, StandardScaler is fitted only on the training set and applied to the testing set.

3. Reproduction Methodology

3.1 Individual Classifiers

Six individual classifiers are reproduced following the model set used in research paper [1]. The research paper does not provide exact hyperparameters, so the default settings from scikit library are applied as reproduction assumptions.

Model	Configuration	Reference
Logistic Regression	sklearn default settings	[5]
Decision Tree	criterion="entropy", random_state=42	[1], [9]
Random Forest	sklearn defaults + random_state=42	[10]
XGBoost	default XGBClassifier + random_state=42	[4]
Gaussian Naive Bayes	default GaussianNB	[2], [6]
KNN	default KNeighborsClassifier (k=5)	[8]

Decision Tree used entropy because the paper's algorithm explicitly describes entropy-based attribute selection [1]. The exact Naive Bayes variant was not reported, so Gaussian Naive Bayes was selected as a reasonable implementation assumption. Other unspecified parameters were kept at their library defaults.

3.2 Stacking Ensemble

Stacking is an ensemble learning technique that combines multiple machine learning models to improve predictive performance. Different base models are trained first, and their predictions are then used as inputs for a higher-level model called a meta-model or meta-learner. The meta-model learns how to combine the outputs of the base models to produce the final prediction [3].

The research paper does not identify exact meta-classifier. Therefore, Logistic Regression is used as the final estimator in the default setting [7]. The six base classifiers used in the individual experiments are retained without changing their configurations.

3.3 Evaluation Metrics

Reproduction models are evaluated by using Accuracy, Precision, Recall, F1 Score and AUC, which are similar to the task requirements and the metrics reported in the selected research paper [1]. Then, confusion matrix and ROC curve are also applied to analyze classification errors and class distinguish.

4. Reproduction Results

4.1 Individual Classifier Results

	Model	Research Paper Accuracy	Research Paper Precision	Research Paper Recall	Research Paper F1	Research Paper AUC	Reproduced Accuracy	Reproduced Precision	Reproduced Recall	Reproduced F1	Reproduced AUC
0	Logistic Regression	0.8439	0.8196	0.9090	0.8620	0.9204	0.7951	0.7563	0.8738	0.8108	0.8787
1	Decision Tree	0.9268	0.9702	0.8909	0.9289	0.9702	0.9854	1.0000	0.9709	0.9852	0.9854
2	Random Forest	0.9268	0.9130	0.9545	0.9333	0.9734	0.9854	1.0000	0.9709	0.9852	1.0000
3	XGBoost	0.9073	0.9174	0.9090	0.9132	0.9830	0.9854	1.0000	0.9709	0.9852	0.9894
4	Naive Bayes	0.8439	0.8250	0.9000	0.8608	0.9185	0.8000	0.7541	0.8932	0.8178	0.8706
5	KNN	0.8585	0.8648	0.8727	0.8687	0.9304	0.8341	0.8000	0.8932	0.8440	0.9486

Figure 5. Published and reproduced performance of the six individual classifiers.

The reproduced Decision Tree, Random Forest and XGBoost models can achieve higher accuracy than the accuracy score in the research paper. Meanwhile, the reproduced Logistic Regression, Naive Bayes and KNN models achieve a lower accuracy than the research paper result. This means that reproduction does not affect all classifiers in the same way. In this, tree-based models can perform better in the reproduced setting, while Logistic Regression and probabilistic or distance-based models remained closer or lower than the research paper result.

4.2 Stacking Ensemble Results

Metric	Published [1]	Reproduction
Accuracy	0.9853	0.9854
Precision	1.0000	1.0000
Recall	0.9727	0.9709
F1	0.9861	0.9852
AUC	0.9880	1.0000

The stacking ensemble shows the closest agreement with the research paper result, which can achieve 98.54% accuracy compared with 98.53% in the research paper. This means that the overall stacking result is successfully reproduced.

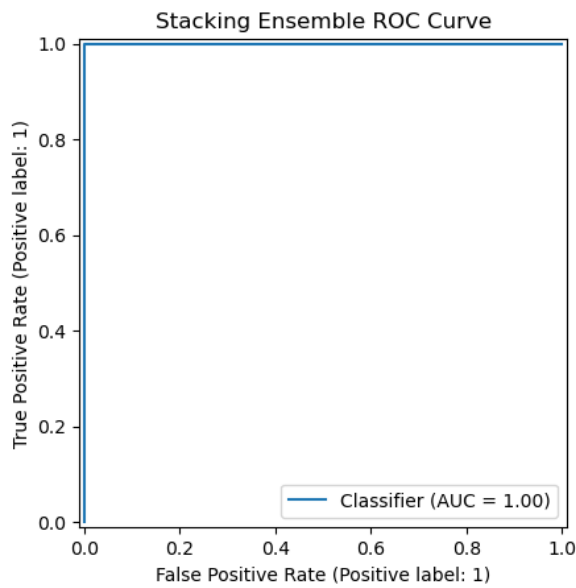


Figure 6. ROC curve of the reproduced stacking ensemble.

The ROC curve produces an AUC of 1.00, which shows very strong class discrimination on the reproduced test set.

4.3 Overall Research Paper against Reproduced Comparison

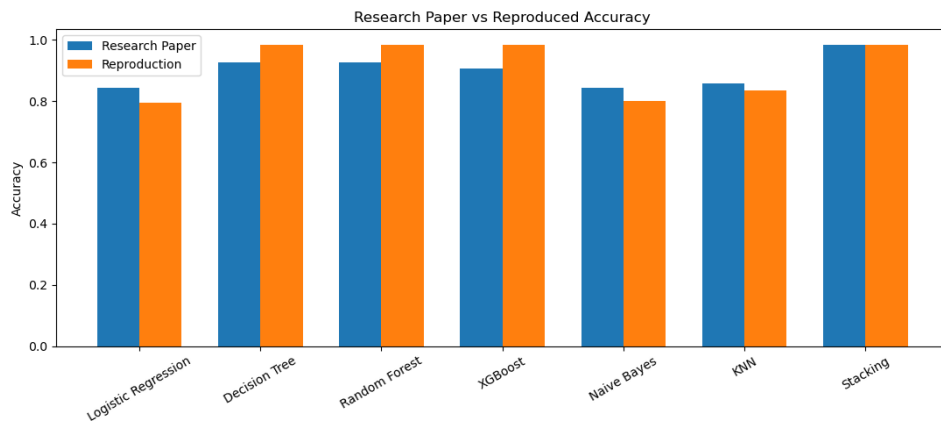


Figure 7. Comparison of published and reproduced accuracy across classifiers.

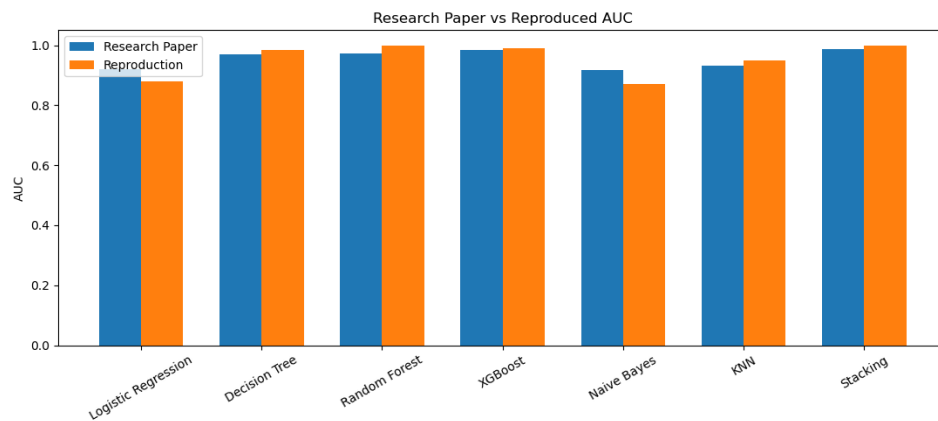


Figure 8. Comparison of published and reproduced AUC across classifiers.

Agreement with the paper varies across classifiers. Logistic Regression and Naive Bayes produce lower accuracy and AUC, while the tree-based models generally produce much higher values. The stacking ensemble demonstrates the closest agreement in accuracy with the research paper. These differences show that only reproduction accuracy alone is not sufficient and that the experimental data structure should also be examined.

5. Critical Analysis

5.1 Duplicated Records and Train-Test Overlap

The original dataset includes 723 duplicated rows. They are still kept in reproduction because the selected research paper reports using all 1025 records and do not describe duplicate removal [1]. Therefore, a diagnostic check is conducted after the random train-test split.

```
Test samples: 205
Test samples also found in training: 199
Percentage of test samples also found in training: 97.07317073170731
```

Figure 9. Train-test duplicate overlap identified in the Part 1 reproduction.

5.2 Unique Dataset Experiment

A supplementary dataset is created by removing exact duplicate rows to investigate the impact of duplication. The dataset decreases from 1025 to 302 unique records. The same 80:20 split, random state, feature scaling and model configurations are still applied.

	Model	Experiment	Accuracy	Precision	Recall	F1	AUC
0	Decision Tree	Research Paper	0.9268	0.9702	0.8909	0.9289	0.9702
1	Decision Tree	Main Reproduction	0.9854	1.0000	0.9709	0.9852	0.9854
2	Decision Tree	Duplicate-Free	0.7213	0.6875	0.7586	0.7213	0.7231
3	Random Forest	Research Paper	0.9268	0.9130	0.9545	0.9333	0.9734
4	Random Forest	Main Reproduction	0.9854	1.0000	0.9709	0.9852	1.0000
5	Random Forest	Duplicate-Free	0.8361	0.7879	0.8966	0.8387	0.8788
6	XGBoost	Research Paper	0.9073	0.9174	0.9090	0.9132	0.9830
7	XGBoost	Main Reproduction	0.9854	1.0000	0.9709	0.9852	0.9894
8	XGBoost	Duplicate-Free	0.8033	0.7576	0.8621	0.8065	0.8524
9	Stacking	Research Paper	0.9853	1.0000	0.9727	0.9861	0.9880
10	Stacking	Main Reproduction	0.9854	1.0000	0.9709	0.9852	1.0000
11	Stacking	Duplicate-Free	0.8361	0.7879	0.8966	0.8387	0.8685

Figure 10. Comparison of published, main reproduction, and duplicate-free diagnostic results.

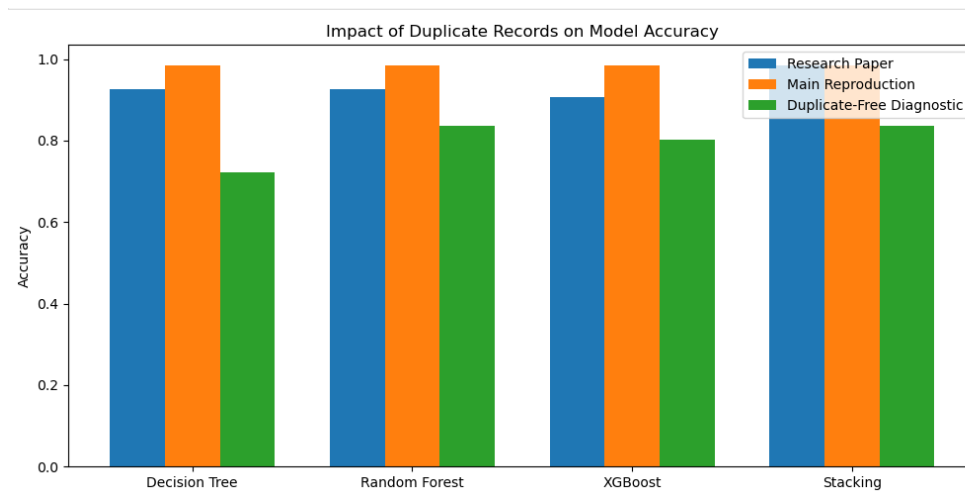


Figure 11. Impact of duplicate removal on model accuracy.

The primary reproduction successfully reproduced the stacking accuracy reported in the paper, which can achieve 98.54% compared with 98.53% [1]. However, the diagnostic analysis shows that the original random split contained duplicate overlaps. After duplicate removal, performance decreases across every investigated model. Decision Tree accuracy decreased from 98.54% to 72.13%, while Random Forest, XGBoost and Stacking decreased to 83.61%, 80.33% and 83.61%, respectively.

These results provide strong evidence that duplicated records in the original dataset can affect performance in the main reproduction. However, the unique experiment is treated only as a diagnostic analysis and does not replace the main reproduction.

6. Proposed Machine Learning Solution

6.1 Motivation and Identified Limitation

The critical analysis in Part 1 identified the significant duplication in the original dataset. In the original dataset, 723 duplicated rows are identified in total 1025 records. Under the reproduced random train-test split, 199/205 test samples had an identical observation in the training set. This overlap reduces the independence of the test set and may expand the measured predictive performance. This concern is supported by the unique diagnostic experiment, where performance of Decision Tree, Random Forest, XGBoost and the stacking ensemble decrease significantly.

Evidence	Result
Original records	1025
Exact duplicate rows	723
Test samples in Part 1	205
Test samples also found in training	199
Exact overlap	97.07%
Duplicate-free records	302

6.2 Proposed Methodology

A Robust Duplicate-Aware Stacking Pipeline is proposed to address the limitation identified in Part 1. The solution changes the data preparation and validation strategy while keeping the original feature set and stacking architecture.

First, exact duplicate values are removed before any model evaluation. This reduces the dataset from 1025 observations to 302 unique observations and prevents identical records from being allocated across training and testing folds.

```
Original records: 1025
Unique records: 302
Removed duplicates: 723
```

Figure 12. Duplicate-removal summary for the proposed method.

Then, the Stratified K-fold maintains similar class distribution across folds [11]. The proposed method extends this evaluation by using RepeatedStratifiedKFold, which repeats stratified K-Fold using different randomized partitions [12].

6.3 Experimental Protocol

Exact duplicates are removed before evaluation, which results in 302 unique observations. The same 13 predictor features and binary target used in Part 1 are kept.

Component	Configuration
Dataset	302 unique records
Predictors	13
Duplicate handling	Duplicates removed before validation
Outer validation	Repeated Stratified K-Fold
Outer folds	5
Repeats	5
Outer evaluations	25
Random state	42
Scaling	StandardScaler inside each pipeline
Base models	Logistic Regression, Decision Tree, Random Forest, XGBoost, Gaussian Nabie Bayes, KNN
Stacking CV	5-fold
Meta-classifier	Logistic Regression
Metrics	Accuracy, Precision, Recall, F1, AUC

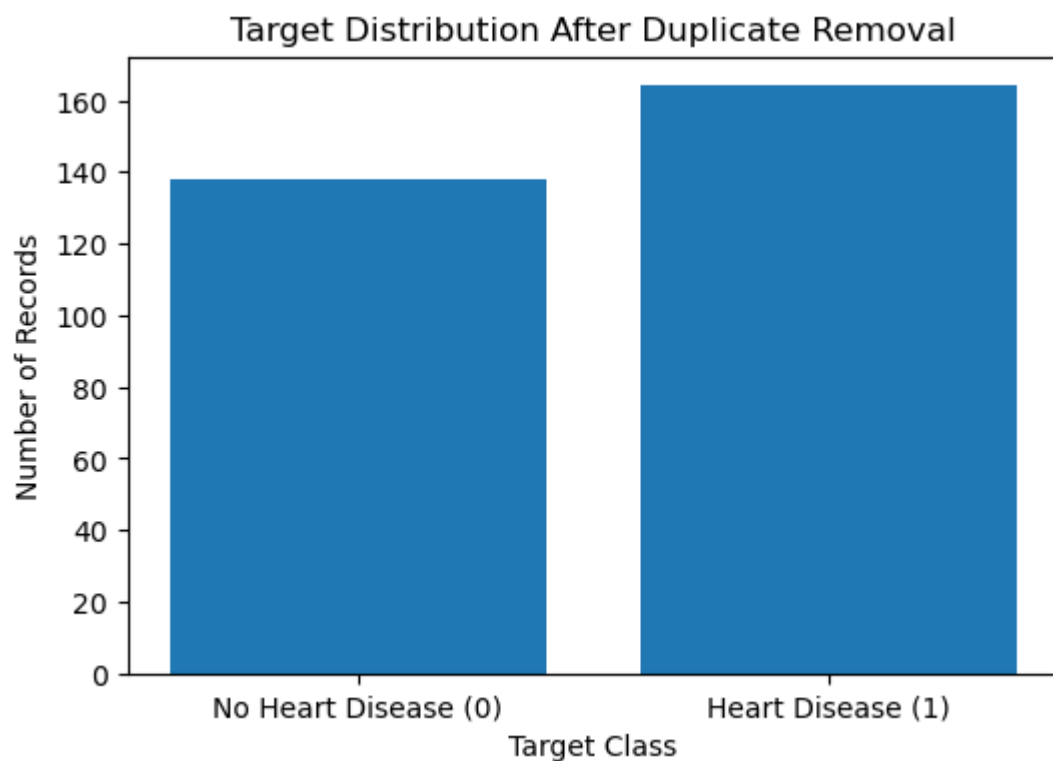


Figure 13. Target-class distribution after duplicate removal.

The duplicate-free dataset remains relatively balanced, with 54.30% positive and 45.70% negative observations.

The same 6 base classifiers and Logistic Regression meta-classifier used in Part 1 are kept isolating the effect of the proposed evaluation methodology. Feature scaling is applied inside each model pipeline so that scaling parameters are estimated only from the corresponding training data rather than from the complete dataset.

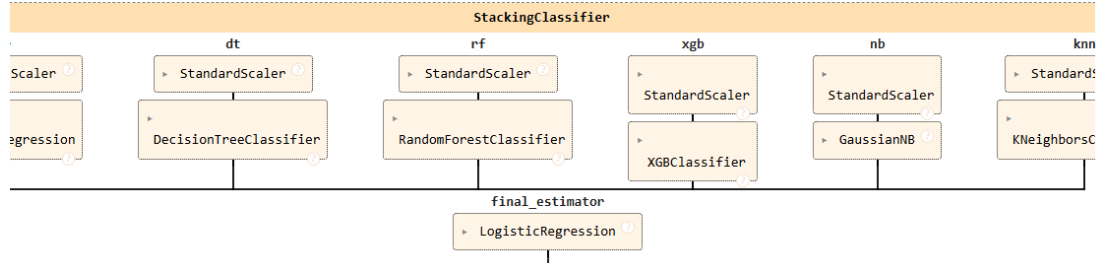


Figure 14. Architecture of the proposed duplicate-aware stacking pipeline.

Then, the outer repeated cross validation evaluates the complete proposed model, where the inner 5-fold procedure is a part of the stacking architecture. Therefore, 2 validation levels serve different purposes.

6.4 Experimental Results

	Repeat	Fold	Accuracy	Precision	Recall	F1	AUC
0	1	1	0.8033	0.8621	0.7576	0.8065	0.8820
1	1	2	0.8033	0.7561	0.9394	0.8378	0.9307
2	1	3	0.8333	0.8438	0.8438	0.8438	0.9096
3	1	4	0.9000	0.8649	0.9697	0.9143	0.9158
4	1	5	0.8167	0.8056	0.8788	0.8406	0.9068
5	2	1	0.8525	0.9000	0.8182	0.8571	0.9426
6	2	2	0.8689	0.8205	0.9697	0.8889	0.9015
7	2	3	0.7833	0.7436	0.9062	0.8169	0.8705
8	2	4	0.8833	0.9062	0.8788	0.8923	0.9080
9	2	5	0.7833	0.7500	0.9091	0.8219	0.8597
10	3	1	0.8525	0.8158	0.9394	0.8732	0.9307
11	3	2	0.8361	0.8485	0.8485	0.8485	0.8929
12	3	3	0.8167	0.8000	0.8750	0.8358	0.8962
13	3	4	0.8000	0.8387	0.7879	0.8125	0.8900
14	3	5	0.7667	0.8065	0.7576	0.7812	0.8855
15	4	1	0.8525	0.8333	0.9091	0.8696	0.9253
16	4	2	0.8361	0.8286	0.8788	0.8529	0.9383
17	4	3	0.8167	0.8000	0.8750	0.8358	0.8873
18	4	4	0.8500	0.8158	0.9394	0.8732	0.9574
19	4	5	0.8167	0.8056	0.8788	0.8406	0.8833
20	5	1	0.8197	0.7895	0.9091	0.8451	0.8788
21	5	2	0.8197	0.7750	0.9394	0.8493	0.8810
22	5	3	0.9167	0.9655	0.8750	0.9180	0.9754
23	5	4	0.8000	0.8621	0.7576	0.8065	0.8597
24	5	5	0.7833	0.7632	0.8788	0.8169	0.9147

Figure 15. Performance results across 25 repeated outer-fold evaluations.

The stacking model shows noticeable performance variation across the 25 outer evaluation folds. Accuracy ranges from around 76.67% to 91.67%, while AUC ranges from around 0.86 to 0.98. Then, precision, recall and f1 score also fluctuate between folds, which shows that the model performance is affected by different training and testing partition after duplicate values are removed.

	Metric	Mean $\pm$ SD
0	Accuracy	0.8284 ± 0.0373
1	Precision	0.8240 ± 0.0519
2	Recall	0.8768 ± 0.0622
3	F1	0.8472 ± 0.0336
4	AUC	0.9050 ± 0.0295

Figure 16. Mean and standard deviation of the proposed method across 25 outer folds.

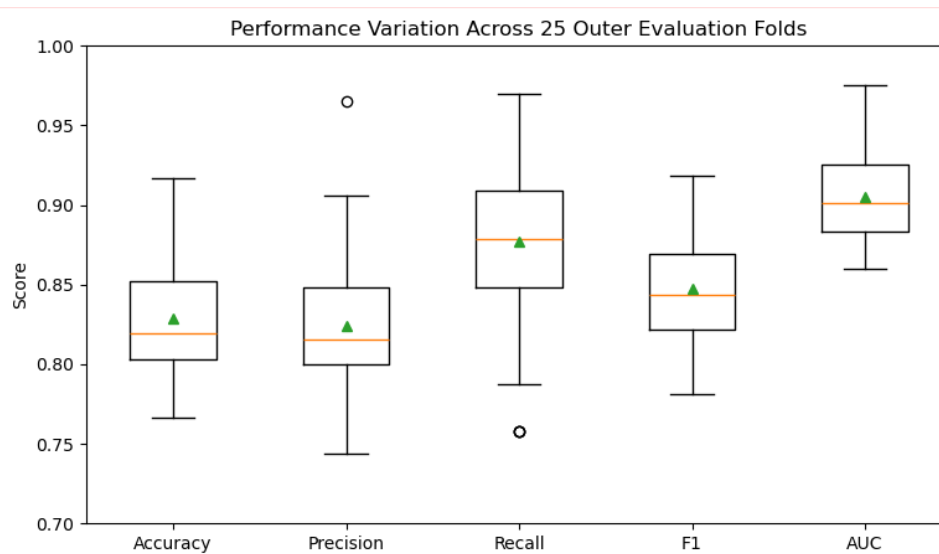


Figure 17. Distribution of evaluation metrics across 25 outer-fold evaluations.

The proposed stacking method can achieve a mean Accuracy at 0.8284 ± 0.0373 , Precision at 0.8240 ± 0.0519 , Recall of $87.68\% \pm 6.22\%$, F1 Score of $84.72\% \pm 3.36\%$, and AUC of $90.50\% \pm 2.95\%$. In this, Recall can achieve the highest average value, which means that the model is effective at identifying positive heart disease cases. Then, the mean AUC score at 0.9050 demonstrates a good overall class distinguish across the repeated evaluations. The standard deviations show that

performance changes between different data partitions, particularly for Precision and Recall. Therefore, reporting the mean and standard deviation across repeated folds provides a more informative estimate of model performance than relying on a single train-test split.

6.5 Comparison

The proposed method is compared with 3 previous results, which results in selected research paper, main reproduction from part 1, unique dataset diagnostic experiment from part 1.

	Method	Accuracy	Precision	Recall	F1	AUC
0	Research Paper	0.9853	1.0000	0.9727	0.9861	0.9880
1	Part 1 Reproduction	0.9854	1.0000	0.9709	0.9852	1.0000
2	Part 1 Unique Dataset Diagnostic	0.8361	0.7879	0.8966	0.8387	0.8685
3	Part 2 Proposed Method	0.8284	0.8240	0.8768	0.8472	0.9050

Figure 18. Comparison of published, reproduced, diagnostic, and proposed results.

The main Part 1 reproduction closely matches the research paper stacking accuracy, with 98.54% compared with 98.53% in the research paper. However, both results are significantly higher than the unique dataset evaluations. The duplicate removed single-split diagnostic achieves 83.61% accuracy, while the proposed repeated evaluation produces a mean accuracy of 82.84%. These similar values provide additional evidence that the lower performance is not limited to one particular unique dataset train-test split. The proposed method achieves slightly lower mean accuracy than the unique dataset diagnostic, but it improves Precision, F1 Score, and AUC. Moreover, it evaluates the model across 25 stratified test folds rather than relying on one random split. Therefore, the main improvement of the proposed method is not a higher raw accuracy than the published result, but a more robust estimate of model generalization across duplicate-free stratified partitions.

6.6 Critical Evaluation of the Proposed Method

The proposed Robust Duplicate-Aware Stacking Pipeline addresses two limitations identified from Part 1.

First, exact duplicate observations are removed before model evaluation. This reduces the dataset from 1025 records to 302 unique records and prevents identical observations from appearing in the training and testing folds.

Second, the original single train-test evaluation is replaced by Repeated Stratified 5-Fold Cross-Validation. Therefore, the model is evaluated across 25 repeated outer folds, which can provide both an average performance estimate and information about variation between different data partitions.

The proposed method achieved an Accuracy of $82.84\% \pm 3.73\%$, Precision of $82.40\% \pm 5.19\%$, Recall of $87.68\% \pm 6.22\%$, F1 Score of $84.72\% \pm 3.36\%$, and AUC of $90.50\% \pm 2.95\%$.

Although these results are significantly lower than the 98.53% accuracy reported in the research paper and the 98.54% Part 1 reproduction, the proposed evaluation does not contain the exact duplicate contamination identified in Part1. Therefore, the lower performance should not be interpreted as a failure of the proposed method. Instead, it provides a more conservative estimate of how the stacking model performs on unique observations.

The variation across the repeated folds also shows that model performance depends on the selected data partition. This supports the use of repeated validation rather than reporting a result from only one random train-test split.

6.7 Limitations of the Proposed Method

The proposed method also has several limitations. First, removing duplicate reduces the dataset from 1025 to only 302 unique records. Therefore, the amount of training data available in each fold is relatively small. Then, repeated cross-validation improves the robustness of performance estimation, but it does not increase the size of the original dataset. After that, the same model configurations from Part 1 are retained to isolate the effect of the proposed evaluation protocol. Therefore, hyperparameter optimization and alternative feature-selection methods are not investigated in this experiment. Finally, the proposed results are obtained from the same public heart disease dataset. Evaluation on an additional independent external dataset would provide stronger evidence of model generalization.

7. Conclusion

This study successfully reproduced the main stacking result reported in the selected research paper, which can achieve 98.54% accuracy compared with the published 98.53%. However, further analysis identified significant duplicate observations and train-test overlap in the original dataset, which may have contributed to the very high performance. Therefore, a Robust Duplicate-Aware Stacking Pipeline was proposed using duplicate removal and Repeated Stratified 5-Fold Cross-Validation. The proposed method achieved a mean Accuracy of 82.84% and AUC of 90.50%. Although the raw performance was lower, the proposed evaluation provides a more robust and conservative estimate of model generalization on unique observations.

8. GitHub and Video Link

GitHub Link: <https://github.com/kent1306/MachineLearningHD>

Video Demonstration: <https://youtu.be/hrbzvywWsoo>

Acknowledgement

I acknowledge that I use ChatGPT during this task to support brainstorming, planning the structure of the experiments, identifying appropriate Python libraries and functions, and assisting with the creation of graphs, tables, and charts. I also used ChatGPT to help review explanations and improve the clarity of my writing. All suggestions generated are reviewed and adapted before included in the final submission. I confirm that I understand the methods, code, experimental results, and analysis presented in this work, and that the final submission represents my own understanding and decisions.

References

- [1] M. Bhagat, A. Sharma, and P. Agarwal, “An Efficient Stacking-Based Ensemble Technique for Early Heart Attack Prediction,” *Multimedia Tools and Applications*, vol. 84, no. 30, pp. 36351–36375, Apr. 2024, doi: [10.1007/s11042-024-19293-7](https://doi.org/10.1007/s11042-024-19293-7).
- [2] GeeksforGeeks, “Naive Bayes Classifiers,” GeeksforGeeks. Accessed: Sep. 27, 2026. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/naive-bayes-classifiers/>
- [3] GeeksforGeeks, “Stacking in Machine Learning,” GeeksforGeeks. Accessed: Sep. 27, 2026. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/stacking-in-machine-learning/>
- [4] GeeksforGeeks, “XGBoost,” GeeksforGeeks. Accessed: Sep. 27, 2026. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/xgboost/>
- [5] scikit-learn, “LogisticRegression,” Scikit-learn.org. Accessed: Sep. 27, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
- [6] scikit-learn, “1.9. Naive Bayes — Scikit-Learn 0.21.3 Documentation,” Scikit-learn.org. Accessed: Sep. 27, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/naive_bayes.html
- [7] scikit-learn, “Sklearn.Ensemble.StackingClassifier — Scikit-Learn 0.23.2 Documentation,” scikit-learn.org. Accessed: Sep. 27, 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html>
- [8] scikit-learn, “Sklearn.Neighbors.KNeighborsClassifier — Scikit-Learn 0.22.1 Documentation,” Scikit-learn.org. Accessed: Sep. 27, 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html>

- [9] scikit-learn, “Sklearn.Tree.DecisionTreeClassifier — Scikit-Learn 0.22.1 Documentation,” Scikit-learn.org. Accessed: Sep. 27, 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>
- [10] scikit-learn, “Sklearn.Ensemble.RandomForestClassifier — Scikit-Learn 0.20.3 Documentation,” Scikit-learn.org. Accessed: Sep. 27, 2026. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>
- [11] GeeksforGeeks, “Stratified K Fold Cross Validation,” GeeksforGeeks. Accessed: Sep. 27, 2026. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/stratified-k-fold-cross-validation/>
- [12] scikit-learn, “Sklearn.Model_Selection.RepeatedStratifiedKFold,” scikit-learn. Accessed: Sep. 27, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.RepeatedStratifiedKFold.html